How Content Delivery Networks (CDNs) Make the Web Fast

Kian Kamali
ENEB352 – Introduction to Networks and Protocols
University of Maryland

Abstract

Content Delivery Networks (CDNs) are distributed server infrastructures designed to deliver web content to users with reduced latency and improved reliability. This report examines the technical mechanisms through which CDNs improve web performance, including the network problems they solve, the protocols they use, their architectural design, and comparisons between major CDN providers. Additionally, this report analyzes the trade-offs involved in CDN deployment and discusses emerging improvements such as edge computing, HTTP/3, and machine learning-driven caching.

1. Introduction

A Content Delivery Network, or CDN, is a geographically distributed network of proxy servers and data centers that work together to deliver internet content rapidly to end users. CDNs cache content at edge locations close to users, which reduces the physical distance data must travel and minimizes the load on origin servers (Nygren et al., 2010). The modern internet depends on CDNs to handle the majority of web traffic, including streaming video, software downloads, and dynamic web applications.

The core purpose of a CDN is to solve the fundamental problem of latency in packet-switched networks. When a user in New York requests a webpage hosted in Los Angeles, the request and response must travel thousands of miles across multiple network hops. Each hop introduces delay, and the cumulative effect creates a poor user experience. CDNs solve this by placing copies of content in data centers around the world, allowing users to retrieve data from the nearest location. This report explains how CDNs achieve these performance gains through specific protocols, architectural decisions, and ongoing technological improvements.

2. The Problem: Latency, Distance, and TCP Overhead

Web performance is limited by three primary factors: propagation delay, network congestion, and protocol overhead. Propagation delay is the time required for a signal to travel across a physical medium, and it is bounded by the speed of light. For example, a signal traveling from New York to London takes approximately 65 milliseconds one way, which means a round trip requires at least 130 milliseconds before any processing occurs (Bocchi et al., 2016). This delay is unavoidable and increases with distance.

Network congestion occurs when intermediate routers and links become saturated with traffic. The Transmission Control Protocol (TCP) uses congestion control algorithms to manage

this, but these mechanisms can slow down data transfer significantly, especially during the initial connection phase. TCP slow start gradually increases the sending rate to avoid overwhelming the network, which means short flows common in web browsing often never reach full speed (Cardwell et al., 2016).

Protocol overhead adds additional latency through connection establishment. A traditional HTTPS request requires a TCP three-way handshake followed by a Transport Layer Security (TLS) handshake. Under TLS 1.2, this process requires two round trips before any application data can be sent. For users far from the origin server, this overhead compounds with propagation delay to create noticeable loading delays.

3. Protocols

3.1 DNS and Anycast

The Domain Name System (DNS) is the first mechanism CDNs use to direct users to the optimal server. When a user types a URL into a browser, a DNS resolver maps the hostname to an IP address. CDNs use DNS-based load balancing to return different IP addresses based on the user's geographic location, network conditions, and server availability (Verma et al., 2017). This allows the CDN to route the user to the closest edge server.

Anycast is a network addressing method that allows multiple servers to share the same IP address. When a user sends a request to an anycast IP address, internet routers direct the packet to the nearest server based on Border Gateway Protocol (BGP) path metrics. This allows CDNs to route traffic efficiently without relying solely on DNS resolution. Cloudflare and other major CDNs use anycast extensively to ensure that user traffic enters their network at the closest point of presence (Sherry et al., 2012).

3.2 Border Gateway Protocol (BGP)

BGP is the routing protocol that controls how packets move between autonomous systems on the internet. CDNs use BGP to advertise their anycast IP addresses from multiple locations, which allows internet service providers to route traffic along the shortest available path. Additionally, CDNs use BGP to handle traffic engineering and failover. If an edge server becomes unavailable, the CDN can withdraw its BGP advertisement, causing traffic to be rerouted to the next nearest location automatically.

3.3 HTTP Evolution and QUIC

The Hypertext Transfer Protocol (HTTP) has evolved significantly to improve performance. HTTP/1.1 allowed persistent connections and pipelining, but it suffered from head-of-line blocking, where a single slow request could delay all subsequent requests on the same connection (Fielding & Reschke, 2014). HTTP/2 introduced multiplexing, which allows multiple requests and responses to be sent concurrently over a single connection, reducing the overhead of establishing multiple TCP connections.

HTTP/3 replaces TCP with QUIC, a transport protocol built on top of User Datagram Protocol (UDP). QUIC reduces connection establishment time by combining the transport and security handshakes into a single exchange. This allows HTTP/3 to send data in zero round-trip time (0-RTT) in some cases, which significantly improves performance for users on high-latency connections (Iyengar & Thomson, 2021). Additionally, QUIC handles packet loss at the application layer rather than the transport layer, which eliminates the head-of-line blocking problems that affect TCP-based protocols.

3.4 TLS 1.3

Transport Layer Security (TLS) 1.3 is the latest version of the cryptographic protocol used to secure HTTPS connections. TLS 1.3 reduces the handshake from two round trips to one, and in cases where the client has previously connected to the server, it allows 0-RTT session resumption (Rescorla, 2018). This reduces the latency penalty of encryption, which is important because modern CDNs serve nearly all content over HTTPS.

4. Architecture

4.1 Cache Hierarchy

CDN architecture is organized in a hierarchical structure. At the top level are origin servers, which store the authoritative copy of all content. Below the origin are regional caches, also known as parent caches or tier-2 caches, which aggregate requests from multiple edge servers. At the bottom level are edge servers, located in points of presence (PoPs) close to end users (Nygren et al., 2010).

When a user requests content, the edge server first checks if it has a cached copy. If the content is not present or has expired, the edge server requests it from a parent cache or directly from the origin. This hierarchical design reduces the load on origin servers and allows popular content to be served entirely from the edge without ever reaching the origin.

4.2 Cache Headers

CDNs use standard HTTP cache headers to determine how long content should be stored and whether it can be reused. The Cache-Control header allows origin servers to specify directives such as max-age, which defines the time in seconds that a resource remains fresh, and immutable, which indicates that the resource will never change and can be cached indefinitely (Fielding & Reschke, 2014). The Expires header provides an absolute expiration date, while ETag and Last-Modified headers allow conditional requests, in which the server returns a full response only if the content has changed.

4.3 Invalidation and Consistency

Cache invalidation is the process of removing or updating cached content when the origin copy changes. CDNs support several invalidation methods. Time-to-live (TTL) expiration is the

simplest approach: cached content is automatically removed after a specified duration. Active invalidation allows origin servers to send purge requests to edge servers to remove specific content immediately. Some CDNs also use cache tagging, which allows groups of related content to be invalidated with a single request.

Maintaining consistency between cached copies and origin data is a fundamental challenge in distributed systems. CDNs use eventual consistency models, which means that updates may not be visible to all users instantly but will propagate across the network over time.

5. Real Systems: Cloudflare, Akamai, and Fastly

5.1 Akamai

Akamai operates one of the largest CDN infrastructures, with approximately 365,000 servers in over 135 countries (Akamai, 2024). Akamai's network is known for its deep penetration into internet service provider networks, which allows it to place servers extremely close to end users. Akamai specializes in video delivery, software downloads, and enterprise security services. The company pioneered many CDN technologies and holds extensive patents in content delivery and distributed caching.

5.2 Cloudflare

Cloudflare operates a global anycast network spanning over 300 cities worldwide (Cloudflare, 2024). Unlike traditional CDNs that charge based on bandwidth usage, Cloudflare offers a free tier with unlimited bandwidth, which has made it popular among small websites and developers. Cloudflare has heavily invested in new protocols, including early support for HTTP/3 and QUIC. The company also provides additional services such as DDoS protection, web application firewalls, and serverless edge computing through Cloudflare Workers.

5.3 Fastly

Fastly focuses on high-performance content delivery for dynamic and programmable content. Fastly's network uses solid-state drives (SSDs) rather than traditional hard drives in its edge servers, which allows for faster cache lookups and better performance for uncacheable content (Fastly, 2024). Fastly is known for its real-time log streaming and edge computing capabilities, which allow developers to run custom logic at the edge using Varnish Configuration Language (VCL). Fastly serves many technology companies and media platforms that require low-latency dynamic content delivery.

6. Trade-offs: Latency vs. Cost vs. Complexity

Deploying a CDN involves balancing three competing factors: latency reduction, financial cost, and operational complexity. Reducing latency requires placing servers in more locations, which increases infrastructure and maintenance costs. For example, Akamai's

extensive network provides excellent performance but comes at a higher price point compared to competitors.

Cost models vary between providers. Some CDNs charge based on bandwidth transferred, while others use request-based pricing or flat monthly fees. For high-traffic websites, bandwidth-based pricing can become expensive quickly. However, the cost of not using a CDN includes higher origin server load, increased bandwidth expenses from the origin data center, and potential revenue loss from users who abandon slow-loading sites.

Complexity is another important consideration. Configuring cache rules, managing SSL certificates across multiple edge locations, and debugging issues in a distributed system require specialized knowledge. Incorrect cache configurations can lead to stale content being served to users or origin servers being overwhelmed by cache misses.

7. Improvements

7.1 Edge Compute

Edge computing extends CDN capabilities by allowing code to execute directly on edge servers rather than at centralized data centers. This reduces latency for applications that require personalization, authentication, or real-time data processing. Cloudflare Workers and Fastly Compute are examples of platforms that allow developers to deploy serverless functions at the edge (Cloudflare, 2024). Edge computing is particularly useful for applications such as A/B testing, bot detection, and dynamic content assembly.

7.2 HTTP/3 and QUIC Adoption

HTTP/3 and QUIC represent the next generation of web transport protocols. As of 2024, major browsers and CDNs have implemented HTTP/3, and adoption continues to grow. These protocols provide particular benefits for mobile users and users on unreliable networks because QUIC handles connection migration gracefully when a device switches between Wi-Fi and cellular networks (Iyengar & Thomson, 2021).

7.3 Machine Learning Caching

Machine learning is increasingly being used to optimize CDN caching strategies. Traditional caching uses fixed TTL values and simple replacement policies such as Least Recently Used (LRU). Machine learning models can predict which content will be requested next based on historical patterns, allowing the CDN to prefetch content before it is requested and to retain popular content longer (Bastug et al., 2017). This approach improves cache hit rates and reduces origin load, particularly for content with predictable access patterns such as video streaming.

8. Conclusion

Content Delivery Networks are essential infrastructure for the modern web, solving fundamental problems of latency and congestion through geographic distribution, intelligent routing, and protocol optimization. By using DNS-based load balancing, anycast routing, and hierarchical caching, CDNs reduce the distance between users and content. Protocol improvements including HTTP/2 multiplexing, TLS 1.3, and QUIC further reduce connection overhead and improve performance on unreliable networks.

Major CDN providers each offer distinct advantages. Akamai provides the largest geographic footprint, Cloudflare offers broad protocol support and accessibility, and Fastly specializes in dynamic content and edge programmability. However, deploying a CDN requires careful consideration of the trade-offs between performance, cost, and complexity.

Looking forward, edge computing, HTTP/3 adoption, and machine learning-driven caching represent significant opportunities for further performance improvements. As web applications become more interactive and real-time, the role of CDNs will continue to expand beyond simple content delivery to encompass computation, security, and intelligent traffic management at the network edge.

References

Akamai. (2024). About Akamai: Our intelligent edge platform. https://www.akamai.com/company/about

Bastug, E., Bennis, M., & Debbah, M. (2017). Living on the edge: The role of proactive caching in 5G wireless networks. IEEE Communications Magazine, 52(8), 82–89. https://ieeexplore.ieee.org/document/6871674

Bocchi, E., De Cicco, L., & Rossi, D. (2016). Measuring the quality of experience of web users. Proceedings of the 2016 ACM on Internet Measurement Conference, 427–434. https://doi.org/10.1145/2987443.2987474

Cardwell, N., Cheng, Y., Gunn, C. S., Yeganeh, S. H., & Jacobson, V. (2016). BBR: Congestion-based congestion control. ACM Queue, 14(5), 20–53. https://doi.org/10.1145/3012426.3022184

Cloudflare. (2024). How Cloudflare works. https://www.cloudflare.com/learning/cdn/what-is-a-cdn/

Fastly. (2024). Fastly network map and points of presence. https://www.fastly.com/network-map/

Fielding, R., & Reschke, J. (2014). Hypertext transfer protocol (HTTP/1.1): Caching (RFC 7234). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7234

Iyengar, J., & Thomson, M. (2021). QUIC: A UDP-based multiplexed and secure transport (RFC 9000). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc9000

Nygren, E., Sitaraman, R. K., & Sun, J. (2010). The Akamai network: A platform for high-performance internet applications. ACM SIGOPS Operating Systems Review, 44(3), 2–19. https://doi.org/10.1145/1842733.1842736

Rescorla, E. (2018). The transport layer security (TLS) protocol version 1.3 (RFC 8446). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc8446

Sherry, J., Hasan, S., Scott, C., Krishnamurthy, A., Ratnasamy, S., & Sekar, V. (2012). Making middleboxes someone else's problem: Network processing as a cloud service. ACM SIGCOMM Computer Communication Review, 42(4), 13–24. https://doi.org/10.1145/2377677.2377680

Verma, D. C., Calo, S., & Amiri, K. (2017). Policy-driven CDN placement in content centric networks. 2017 IEEE International Conference on Communications (ICC), 1–6. https://doi.org/10.1109/ICC.2017.7996631